자기소개서 — 유경동 (Unity · C# 게임 클라이언트)

1. 라이브 이벤트의 보상·저장·광고 복귀 경로를 구현했습니다

2025년 7월부터 씨플레이에서 모바일 게임 CatJump의 Unity·C# 클라이언트를 개발하고 있습니다. CatJump는 Google Play 다운로드 500만 이상, 약 70만 MAU 규모로 운영 중인 게임입니다. 저는 약 2~3주 주기 업데이트에 맞춰 기간 한정 이벤트, 보상·스킨 같은 메타 기능, 10인 실시간 PvP를 맡아 왔습니다.

이벤트 하나를 내보내려면 플레이 루프 외에도 진입 조건, 결과 정산, 보상 지급과 저장, 광고를 보고 돌아온 뒤의 상태, 기간이 끝난 뒤의 정리가 함께 맞아야 합니다. 제가 수정한 오류 중에도 이 경로에서 나온 것이 있습니다. 광고 복귀 뒤 보상 저장이 빠지는 경로, PvP의 룸 진입 크래시와 로비 복귀 버그가 그 예입니다. 신규 미니게임은 매번 독립 시스템으로 만들지 않고 공통 이벤트 흐름(진입·결과·보상·저장·기간 종료)에 연결했습니다.

보상 상자에서는 표시 계층이 확정된 결과만 받도록 지급·저장과 팝업 표시를 나눴고, 추첨 전에 획득할 수 없는 후보를 제외했습니다. 아래 두 사례도 기준이 되는 상태와 화면에 보여 주는 상태, 정상 흐름과 예외 흐름을 나눠 작업한 기록입니다. 라이브 이벤트의 보상·저장·UI 상태 연결과 Photon 기반 멀티플레이를 구현한 경험을 Unity 클라이언트 업무에 적용하고자 합니다.

2. 10인 PvP의 호스트 변경 복구와 순번 카운트다운

CatJump의 10인 실시간 점프 레이스를 Photon PUN2로 구현했습니다. 경기 흐름 설계부터 출시 후 오류 수정까지 제가 맡았습니다. 전용 서버 없이 마스터 클라이언트가 진행 권한을 갖는 구조이고, 플레이 환경은 모바일 네트워크입니다. 이 구조에서는 마스터가 경기 중에 나가면 준비·카운트다운·결과 처리가 멈추거나 클라이언트마다 다르게 진행될 수 있었습니다. 또 이벤트를 받는 시점이 클라이언트마다 달라 늦게 도착한 카운트다운이 현재 상태를 덮어쓸 수 있고, 경기 중 이탈로 인원이 줄면 레이스의 경쟁감이 떨어지는 문제도 있었습니다.

먼저 경기를 매칭 탐색, 인원 확정, 준비·카운트다운, 플레이, 결과 단계로 나눴습니다. 경기 진행 단계의 복구 기준은 룸 속성에 두었습니다. 새 마스터는 이 값을 읽어 경기를 재개하고, 각 클라이언트의 단계 표시도 같은 값을 따르도록 했습니다. 카운트다운 이벤트에는 순번을 붙여, 이미 지난 순번이 늦게 도착하면 반영하지 않게 했습니다. 경기 중 이탈한 유저의 자리는 AI로 대체해, 남은 참가자의 레이스가 끊기지 않게 만들었습니다.

출시 후 발생한 카운트다운 불일치, 룸 진입 크래시, 로비 복귀 버그를 수정했습니다.

3. 이벤트마다 하드코딩하던 대사·연출을 공용화했습니다

CatJump의 기간 한정 이벤트에는 캐릭터 대사와 연출이 자주 들어갑니다. 공용 시스템이 생기기 전에는 이벤트마다 대사와 연출을 코드에 직접 작성했습니다. 이 방식에서는 새 이벤트를 만들 때마다 대사 UI 연결, 종료 처리, 예외 처리를 다시 구현해야 했고, 같은 종류의 결함이 이벤트마다 따로 생길 수 있었습니다. 이벤트가 약 2~3주 주기로 추가되는 운영이라 이 반복도 계속됐습니다.

공용 대사 시스템은 설계와 구현을 처음부터 맡았습니다. 대사 내용과 순서는 시트에서 만든 데이터 자산으로 관리하고, 대사 진행 흐름과 UI 표시는 서로 다른 책임으로 나눴습니다. 이벤트마다 반복되던 처리는 시스템 쪽에 한 번만 두었습니다. 리치 텍스트를 한 글자씩 보여 줄 때 태그가 깨지지 않게 하는 처리, 뒤로가기로 닫기, 대사가 끝난 뒤 다음 동작을 콜백으로 넘기는 연결이 여기에 해당합니다. 이미 하드코딩으로 만든 기존 이벤트와 함께 동작해야 했고, 현지화 리소스와도 맞물려야 했습니다.

이후 여러 이벤트에서 이 공용 대사 시스템을 재사용했습니다.

4. 예외 경로를 QA 확인 항목으로 정리하고 레벨 에디터를 맡았습니다

이탈, 재진입, 광고 복귀, 중복 수령, 기간 종료 같은 예외 경로는 재현 가능한 케이스로 정리해 QA 확인 항목으로 넘겼습니다. 메인 화면에서 광고를 보고 받는 보상의 저장이 빠지는 경로를 수정하면서, 광고 결과 이후의 지급·저장·화면 갱신 순서를 확인했습니다. 개발 환경에서 재현되지 않는 크래시는 Crashlytics 리포트로 발생 단계와 데이터 조건을 좁혀 재현·확인 항목을 정리했습니다.

기획자와 맞닿는 작업으로는 레벨 에디터 업데이트를 맡았습니다. 새 인게임 오브젝트를 레벨 에디터가 지원하지 않으면 기획자가 스테이지에 쓸 수 없고 개발자 손을 다시 거쳐야 합니다. 스킨 리소스를 하나씩 수동 등록하던 과정을 폴더를 선택해 데이터를 추가하는 에디터 도구로 바꾸는 작업에도 참여했습니다.

이벤트가 쌓이면 끝난 이벤트의 자산이 빌드에 남고, 테스트용 치트 기능이 라이브 빌드에 섞일 위험도 생깁니다. 릴리즈 전에는 종료된 이벤트 자산을 지우고 남은 참조를 확인했고, 테스트용 치트 기능은 에디터에서만 동작하도록 제한했습니다. 기능을 고칠 때는 텍스트·이미지 현지화 리소스를 같이 반영했습니다.

5. 실무에서 맡아 온 범위와 공개 저장소에서 확인할 수 있는 작업

실무에서 맡아 온 범위는 라이브 이벤트의 진입·보상·저장·기간 종료, 화면 간 UI 상태 연결, Photon 기반 이탈·호스트 변경 처리입니다. 서비스 중인 기능 위에 새 기능을 붙이는 일도 해 왔습니다. CatJump에서는 이미 쓰이던 캐릭터 색상 선택 상태를 보존하면서 스킨 보유·선택 상태를 수집·프로필·PvP 결과 화면에 연결했습니다.

반복되는 구현을 공용 구조로 바꾸는 작업도 해 왔고, 앞의 공용 대사 시스템이 그 예입니다. 회사 코드는 공개할 수 없습니다. 대신 팀 프로젝트에서 제가 커밋한 변경은 GitHub에서 확인할 수 있습니다. DARK SHIELD에서는 스테이지 전체를 미리 만들던 방식을 현재 스테이지의 다음 후보만 만드는 방식으로 바꿨습니다. Jumac에서는 점수를 키로 쓰던 정렬 사전을 목록 정렬로 바꿔 동점 기록이 사라지지 않게 했습니다.

C·C++ 기초는 42서울 2년 과정과 소켓·kqueue를 사용한 IRC 서버 팀 프로젝트에서 다뤘습니다. 서울·경기·부산 모두 근무할 수 있고, 수도권으로 이주할 수 있습니다.